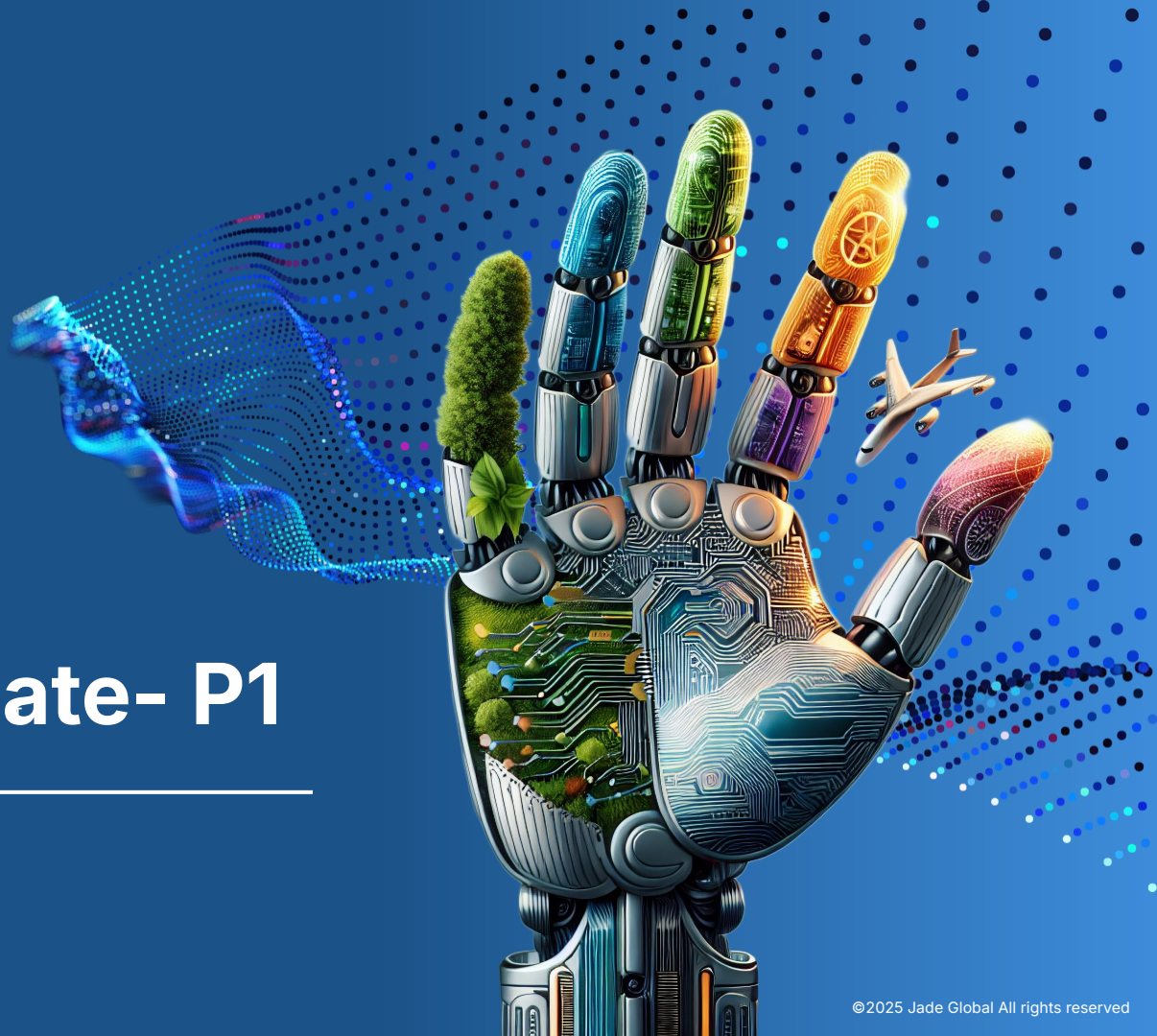
JADE™ GLOBAL

LLM Intermediate- P1

Prompt engineering,
LLM APIs,
Langchain Basics



©2025 Jade Global All rights reserved

Basics Recap

Everything you need to remember from LLM Basics

01

Intro to AI

AI ⟶ ML ⟶ DL ⟶ LLMs. Neural nets learn from data.

02

NLP Fundamentals

Tokenization, embeddings, language modeling.

03

What are LLMs

Large scale text-trained models: GPT, Claude, Gemini.

04

Transformers

Attention mechanism, encoder-decoder, positional encoding.

05

LLM Applications

Text gen, code, summarization, QA, multimodal.

06

Prompt Basics

Zero-shot, few-shot, chain-of-thought prompting.

What We'll Cover Today

Your roadmap for LLM Intermediate – Part 1

01 Prompt Engineering

Advanced techniques to get the best from LLMs

- Role prompting
- ReAct & self-ask
- Prompt chaining
- System prompts

02 LLM APIs

Integrate LLMs into your code

- OpenAI & Anthropic API
- Request parameters
- Streaming responses
- Error handling

03 LangChain Basics

Framework for building LLM-powered apps

- Chains & agents
- Memory management
- Tools & callbacks
- LCEL syntax

MODULE 01

Prompt Engineering

From basic prompts to advanced strategies

Why Prompt Engineering Matters

The difference between good and great LLM output

❌ Weak Prompt

Summarize this article.

LLM Output:

"The article talks about many things related to the topic and contains various information that is useful for readers interested in learning about it..."

Problems:
- Too vague — no length, format or audience
- No context about article type
- Generic, unhelpful output

✅ Strong Prompt

"Summarize this tech article in 3 bullet points for a non-technical manager. Focus on business impact."

LLM Output:

"• Reduces operational cost by 30%
• Automates customer support at scale
• No technical expertise required to deploy"

Why it works:
- Clear audience & purpose
- Specified format & length
- Defined focus area

Role Prompting & System Prompts

Give the model a persona to unlock better responses

💡 *Role prompting: "You are a [role]. [task]" — Assigning a role sets tone, expertise & style for all responses.*

Senior Python Dev

"You are a senior Python developer. Review this code for performance issues."

→ The model now responds as this expert!

Medical Expert

"You are a medical professional. Explain this diagnosis in plain English for a patient."

→ The model now responds as this expert!

Socratic Tutor

"You are a Socratic tutor. Do not give answers — only ask guiding questions."

→ The model now responds as this expert!

System Prompt = Persistent role instruction sent before every user message. Used in chat APIs to define behavior globally.

Advanced Prompting Strategies

Chain-of-Thought, Self-Ask & ReAct

Chain-of-Thought (CoT)

Force reasoning by asking for step-by-step thinking.

```
"Solve this problem step by step:
A train travels 60 km/h...

Step 1: Calculate time...
Step 2: ...
Answer: 2 hours"
```

Best for: Math, logic, multi-step problems

Self-Ask Prompting

Model asks itself sub-questions to break down complex queries.

```
"Q: Who was president when Google
was founded?
Sub-Q: When was Google founded? →
1998
Sub-Q: Who was US president in
1998? → Clinton
Answer: Bill Clinton"
```

Best for: Research, fact-heavy questions

ReAct Prompting

Reasoning + Acting: model thinks, takes action, observes, repeats.

```
"Thought: I need to find today's
weather.
Action: search[weather Mumbai
today]
Observation: 32°C, humid
Thought: Now I can answer.
Answer: It is 32°C in Mumbai."
```

Best for: Agents, tool use, dynamic tasks

Prompt Chaining & Templates

Breaking complex tasks into connected steps

Step 1
Extract Facts

Step 2
Analyze Sentiment

Step 3
Generate Report

Step 4
Format Output

Output of each step → Input of next step. Enables complex multi-stage workflows.

```
Prompt Template (Python f-string):


def summarize_prompt(article, audience, length):
    return f"""You are a professional editor.
    Summarize the following article for {audience}.
    Keep it under {length} words. Focus on key insights.
    Article: {article}
    Summary:"""

# Chain: article → extract_facts() → summarize() → format_as_pdf()
```

Prompt Anti-Patterns

Common mistakes that break your prompts

❌ Negative Instructions Only

Bad: "Don't make it too long"

Fix: "Respond in exactly 3 bullet points, max 15 words each"

❌ Ambiguous Pronouns

Bad: "Take the report, analyze it, then improve it"

Fix: "Analyze the sales report for trends. Then rewrite the Executive Summary section."

❌ Conflicting Instructions

Bad: "Be concise but also very thorough and detailed"

Fix: "Write a 3-sentence summary, then a 2-paragraph deep-dive"

❌ No Output Specification

Bad: "Tell me about Python"

Fix: "List 5 Python strengths as a numbered list with one example each"

MODULE 02

LLM APIs

Integrating language models into your applications

What is an LLM API?

Your bridge between code and language models

Your App
(Python / JS)

HTTP Request

(JSON payload)

LLM API
(OpenAI / Anthropic)

Response

(JSON + text)

```
Python – OpenAI API example:

import google.generativeai as genai

genai.configure(api_key="YOUR_API_KEY")

model = genai.GenerativeModel("gemini-2.0-flash")

response = model.generate_content("Explain tokenization in simple terms")

print(response.text)
```

Key API Parameters

Tuning the model's behavior for your use case

`temperature` *0.0 – 2.0*

Controls randomness. 0 = deterministic, 1+ = creative.

e.g. 0.0 for facts, 0.7 for writing, 1.2 for brainstorming

`max_tokens` *1 – 128k+*

Maximum tokens in the response.

e.g. 100 for short answers, 4000 for long reports

`top_p` *0.0 – 1.0*

Nucleus sampling — only considers top-p probability mass.

e.g. Use with temperature. Typically leave at 1.0

`system` *string*

Persistent instruction prepended to every conversation turn.

e.g. "You are a helpful coding assistant."

`stream` *true / false*

Returns tokens as they generate (real-time output).

e.g. true for chatbots, false for batch processing

`stop` *string[]*

Sequences at which to stop generating.

e.g. ["\n", "###"] to stop at newlines or markers

Roles, Messages & Conversation History

How the chat format structures LLM interactions

`role: "system"`

Sets the model's persona, rules, and behavior. Sent once at the start.

```
"You are a helpful, concise assistant for a medical startup."
```

Keep in messages[] array to maintain context!

`role: "user"`

The human's turn. Contains questions, instructions, or data.

```
"What are the symptoms of Type 2 Diabetes?"
```

Keep in messages[] array to maintain context!

`role: "assistant"`

The model's response. Include in history for multi-turn conversations.

```
"Type 2 Diabetes symptoms include frequent urination, fatigue..."
```

Keep in messages[] array to maintain context!

Error Handling & Rate Limits

JADE
GLOBAL

Writing production-ready LLM integrations

Common Errors

401	**`AuthenticationError`** Invalid API key — check your env vars
429	**`RateLimitError`** Too many requests — implement exponential backoff
400	**`BadRequestError`** Malformed payload — check message format
500	**`InternalServerError`** Provider-side issue — retry with backoff
context	**`ContextLengthExceeded`** Prompt too long — chunk your input

Exponential Backoff Pattern:

```
import time, openai

def call_with_retry(prompt, retries=3):
    for attempt in range(retries):
        try:
            return client.chat.completions.create(
                model="gpt-4o",
                messages=[{"role":"user",
                           "content": prompt}]
            )
        except openai.RateLimitError:
            wait = 2 ** attempt  # 1, 2, 4 sec
            time.sleep(wait)
    raise Exception("Max retries exceeded")
```

MODULE 03

LangChain Basics

Building powerful LLM-powered pipelines

What is LangChain?

The framework for building production LLM apps

LangChain is an open-source framework that lets you build applications powered by LLMs — combining prompts, memory, tools, and external data into pipelines.

🤖 LLMs / Chat Models

Wraps OpenAI, Anthropic, HuggingFace etc. in a unified interface.

📝 Prompt Templates

Reusable parameterized prompts. Fill variables at runtime.

🔗 Chains

Sequence multiple steps: prompt → LLM → parser → next step.

🧠 Memory

Persists conversation context across multiple LLM calls.

🕵️ Agents

LLM decides which tools to use and in what order.

🔧 Tools & Retrievers

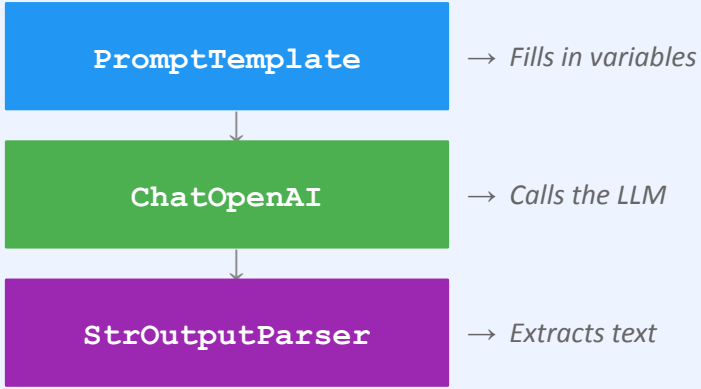
Search, APIs, vector DB lookups plugged into your pipeline.

Chains & LCEL Syntax

Composing LLM pipelines with LangChain Expression Language

What is LCEL?

LangChain Expression Language (LCEL) uses the | pipe operator to chain components together — like Unix pipes for LLMs.




```
LCEL chain example:


from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import
StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in one sentence."
)
llm = ChatOpenAI(model="gpt-4o")
parser = StrOutputParser()

# LCEL pipe syntax
chain = prompt | llm | parser

# Run the chain
result = chain.invoke({"topic": "transformers"})
print(result)
# → "Transformers use attention to process..."
```

Memory in LangChain

Giving LLMs the ability to remember past turns

📋 ConversationBufferMemory

Stores the full chat history verbatim. Simple but gets long.

✅ Easy to use

⚠️ Uses lots of tokens

```
memory = Conversation
BufferMemory()
chain = ConversationChain
(llm=llm, memory=memory)
```

📝 ConversationSummaryMemory

Summarizes past turns as the chat grows. Saves token budget.

✅ Token efficient

⚠️ May lose detail

```
memory = Conversation
SummaryMemory(llm=llm)
chain = ConversationChain
(llm=llm, memory=memory)
```

🔍 VectorStoreRetrieverMemory

Stores memories in a vector DB, retrieves relevant ones per turn.

✅ Scales to long chats

⚠️ More complex setup

```
memory = VectorStore
RetrieverMemory(
retriever=vectorstore
.as_retriever())
```

LangChain Agents & Tools

When the LLM decides what to do next

The Agent Loop

User Input → **LLM Thinks (ReAct)** → **Selects Tool** → **Tool Result**

↑ *repeat until final answer*

🔍 **SerpAPI / TavilySearch**

Live web search

🐍 **PythonREPLTool**

Execute Python code

📖 **WikipediaQueryRun**

Query Wikipedia

🗃️ **SQLDatabaseTool**

Run SQL queries

⚙️ **Custom Function Tool**

Wrap any Python fn as tool

👤 **HumanInputRun**

Ask a human for clarification

KNOWLEDGE CHECK

Quick Quiz — Intermediate P1

1. What is the difference between zero-shot and role prompting?

2. Name 2 API parameters and explain when you would change them.

3. Explain what LCEL is and write the pipe syntax for a simple chain.

4. What type of LangChain memory would you use for a long customer support chat and why?

Part 1 Complete!

Modules covered in Intermediate P1

✓ **01**

Prompt Engineering

✓ **02**

LLM APIs

✓ **03**

LangChain Basics

Coming Up in Intermediate P2 🚀

04 Embeddings

Converting text to vector representations

05 Vector Databases

Storing & searching embeddings at scale

06 RAG

Retrieval Augmented Generation — grounding LLMs in your data

©2025 Jade Global. All rights reserved.

JADE™ GLOBAL

LLM Intermediate - P2

Embeddings,
Vector Databases,
Multi agent systems.

©2025 Jade Global All rights reserved

What We'll Cover Today

Your roadmap for LLM Intermediate – Part 2

04 Embeddings

Text as mathematical vectors

- Word & sentence embeddings
- Semantic similarity
- Embedding models
- Cosine similarity

05 Vector Databases

Store and search at scale

- How vector DBs work
- Pinecone, Weaviate, Chroma
- Indexing strategies
- ANN search

06 RAG

Ground LLMs in your own data

- RAG pipeline overview
- Chunking strategies
- Retrieval & reranking
- End-to-end RAG app

MODULE 04

Embeddings

Turning text into numbers that capture meaning

What are Embeddings?

Representing language as points in mathematical space

💡 *An embedding is a list of numbers (a vector) that represents the meaning of a word, sentence or document. Similar meanings → similar vectors.*

"king"	"queen"	"apple"
▼	▼	▼
`[0.82, 0.41, 0.93, ...]`	`[0.81, 0.43, 0.90, ...]`	`[0.12, 0.77, 0.04, ...]`

← similar vectors → *far apart →*

Famous example: king − man + woman ≈ queen

Embeddings capture relationships between concepts as arithmetic in vector space — this is the foundation of semantic search, RAG, and more.

Types of Embeddings

From words to documents — choosing the right granularity

📝 Word Embeddings

Each word gets its own vector. Classic models.

Models: Word2Vec, GloVe, FastText

✅ Fast, lightweight

⚠️ No context — 'bank' (river) = 'bank' (finance)

💬 Sentence Embeddings

Entire sentence encoded as a single vector.

Models: SBERT, all-MiniLM-L6-v2

✅ Captures sentence-level meaning

⚠️ Loses word-level nuance

🧠 Contextual Embeddings

Each word's vector depends on its full context.

Models: BERT, RoBERTa, OpenAI ada-002

✅ Handles polysemy, state-of-the-art

⚠️ Slower, more compute

📄 Document Embeddings

Entire document or paragraph as one vector.

Models: Doc2Vec, text-embedding-3-large

✅ Good for long-doc retrieval

⚠️ Loses fine-grained detail

Embedding Models & APIs

How to generate embeddings in practice

Model	Provider	Dimensions	Best For	Cost
text-embedding-3-small	OpenAI	1,536	General purpose, fast	Low
text-embedding-3-large	OpenAI	3,072	High accuracy tasks	Medium
all-MiniLM-L6-v2	HuggingFace	384	Local / free	Free
embed-english-v3.0	Cohere	1,024	Multilingual, production	Medium
nomic-embed-text	Nomic	768	Open-source, on-premise	Free

```
Generate embeddings with OpenAI:

from openai import OpenAI
client = OpenAI()
response = client.embeddings.create(model="text-embedding-3-small", input="What is machine learning?")
vector = response.data[0].embedding   # list of 1536 floats
```

Cosine Similarity & Semantic Search

How we measure meaning-closeness between vectors

```
cosine_similarity(A, B)  =  (A · B) / (|A| × |B|)     →  range: -1
                                (opposite) to +1 (identical)
```

−1.0	→	0.0	→	0.7+	→	1.0
Opposite		Unrelated		Similar		Identical

```
Semantic search in Python:
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

query_vec   = embed("What is deep learning?")          # shape: (1, 1536)
doc_vecs    = embed(["Intro to neural nets", "Python basics", "DL overview"])

scores = cosine_similarity([query_vec], doc_vecs)[0]   # [0.91, 0.12, 0.88]
best_match = docs[np.argmax(scores)]                    # "DL overview"
```

MODULE 05

Vector Databases

Storing and searching millions of embeddings at scale

Why Vector Databases?

Regular databases can't do semantic search

❌ Traditional Database

```
SELECT * FROM docs WHERE text LIKE '%machine learning%'
```

- • Keyword matching only — misses synonyms
- • No understanding of meaning
- • Slow brute-force scan at scale
- • Can't rank by semantic relevance

✅ Vector Database

```
db.query(embed('machine learning'), top_k=5)
```

- • Semantic similarity search (meaning, not keywords)
- • Returns ranked results by relevance score
- • ANN search — millions of vectors in milliseconds
- • Supports metadata filtering alongside vector search

Popular Vector Databases

Choosing the right tool for your use case

Pinecone — *Managed Cloud*

✅ Fully managed, easy to start

⚠️ Paid, vendor lock-in

Use: Production apps, fast setup

Weaviate — *Open Source / Cloud*

✅ Hybrid search, GraphQL

⚠️ More complex setup

Use: Enterprise, hybrid search

Chroma — *Open Source / Local*

✅ Simple, Python-native, free

⚠️ Not production-scale yet

Use: Prototyping, LangChain dev

Qdrant — *Open Source / Cloud*

✅ Fast, rich filtering, Rust-based

⚠️ Newer, smaller community

Use: High-performance production

FAISS — *Library (Meta)*

✅ Blazing fast, battle-tested

⚠️ In-memory only, no built-in API

Use: Research, on-prem, custom

pgvector — *PostgreSQL Extension*

✅ Already in your Postgres DB

⚠️ Slower than dedicated DBs

Use: Existing Postgres projects

How Vector DBs Work

Indexing strategies & Approximate Nearest Neighbor search

Indexing Strategies

Flat (Brute Force)

Compare query to every vector. 100% accurate but slow at scale.

📌 *Good for < 10k vectors*

IVF (Inverted File)

Clusters vectors; only search nearby clusters. Faster with small accuracy trade-off.

📌 *Good for 100k–10M vectors*

HNSW

Hierarchical Navigable Small World graph. Fast + accurate. Industry standard.

📌 *Best overall for production*

```
Chroma — upsert & query example:

import chromadb
client = chromadb.Client()
collection = client.create_collection("my_docs")
collection.add(documents=["ML is cool", "Python rocks"], ids=["doc1","doc2"])
results = collection.query(query_texts=["machine learning"], n_results=2)
print(results["documents"])   # [['ML is cool', 'Python rocks']]
```

Metadata Filtering & Hybrid Search

Combining vector search with structured filters

💡 *Vector search finds semantically similar documents. Metadata filtering narrows results by structured fields (date, category, author). Hybrid = both at once.*

Metadata Filtering

```
collection.query(
  query_texts=["AI trends"],
  where={"category": "tech",
         "year": {"$gte": 2023}},
  n_results=5
)
```

Filter by structured fields before or after vector ranking.

Hybrid Search (BM25 + Vector)

```
# Weaviate hybrid example
client.query.get("Article")
  .with_hybrid(query="LLM best practices",
               alpha=0.5)  # 0=BM25, 1=vector
  .with_limit(5).do()
```

Alpha controls the blend: 0 = pure keyword, 1 = pure semantic.

MODULE 06

Retrieval Augmented Generation

Grounding LLMs in your own real-world data

JADE™ GLOBAL

What is RAG?

Teaching an LLM about your data without retraining it

❌ The Problem

LLMs only know what they were trained on (cutoff date). They hallucinate when asked about your private docs, recent events, or proprietary data.

✅ RAG Solution

Retrieve relevant documents at query time, inject them into the prompt as context. The LLM answers using real, up-to-date information.

The RAG Pipeline

User Query → Embed Query → Search Vector DB → Retrieve Top-K Docs → Prompt + Context → LLM Answer

RAG vs Fine-tuning: RAG is preferred when data changes frequently or is large. Fine-tuning is better for teaching the model a new style or format. RAG is cheaper and faster to update.

Document Ingestion & Chunking

How to prepare your documents for RAG

💡 *Chunking = splitting a large document into smaller pieces before embedding. The size and overlap of chunks dramatically affects retrieval quality.*

Fixed-Size Chunks

Split every N characters or tokens, with an overlap window.

```
splitter = CharacterTextSplitter(
  chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_text(doc)
```

📌 *Simple. Good starting point for any text.*

Recursive Character Split

Tries to split at paragraphs → sentences → words.

```
splitter =
RecursiveCharacterTextSplitter(
  chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_text(doc)
```

📌 *Best default. Preserves natural structure.*

Semantic Chunking

Splits where embedding similarity drops — keeps related ideas together.

```
splitter = SemanticChunker(
  embeddings=embed_model)
chunks = splitter.split_text(doc)
```

📌 *Highest quality. More expensive to compute.*

Retrieval Strategies & Reranking

Getting the most relevant context to your LLM

Similarity Search (Top-K)

Return top-K most similar chunks by cosine similarity. Fast, simple baseline.

Use when: Starting point for any RAG system

MMR (Max Marginal Relevance)

Balances relevance AND diversity — avoids returning 5 near-identical chunks.

Use when: When results tend to be redundant

Multi-Query Retrieval

LLM generates N paraphrases of the query; retrieves for all, then deduplicates.

Use when: Ambiguous or complex queries

Cross-Encoder Reranking

After initial retrieval, a reranker scores (query, doc) pairs for precision.

Use when: High-stakes answers requiring accuracy

End-to-End RAG with LangChain

Putting it all together in code

1. **Load Documents**
2. **Split into Chunks**
3. **Embed & Store**
4. **Retrieval Chain**
5. **LLM Answer**

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA

# 1. Load docs
docs = PyPDFLoader("company_policy.pdf").load()
# 2. Chunk
chunks = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
).split_documents(docs)
# 3. Embed & Store
vectorstore = Chroma.from_documents(
    chunks, OpenAIEmbeddings())
# 4 & 5. Retrieve + Answer
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o"),
    retriever=vectorstore.as_retriever(
        search_kwargs={"k": 4})
)
print(qa_chain.invoke("What is the vacation policy?"))
```

RAG Evaluation & Common Problems

How to measure and improve your RAG pipeline

Key RAG Metrics

Metric	Measures	Good Score	Tool
Context Precision	Are retrieved docs actually relevant?	> 0.8	RAGAs
Context Recall	Did we retrieve all needed docs?	> 0.7	RAGAs
Faithfulness	Does answer stick to retrieved context?	> 0.9	RAGAs / TruLens
Answer Relevance	Does the answer address the question?	> 0.8	RAGAs

Common RAG Problems & Fixes

❌ **Wrong chunks retrieved**

✅ Adjust chunk size / add metadata filters

❌ **Hallucination despite context**

✅ Add system prompt: 'Only answer from context. Say I don't know if unsure.'

❌ **Slow retrieval**

✅ Switch from Flat to HNSW index, add caching

Full Stack: Embeddings + VectorDB + RAG

How all three modules connect in a real application

OFFLINE (Index Time) | **ONLINE (Query Time)**

Raw Docs (PDF/Web/DB) → Chunk (LangChain) → Embed (OpenAI) → Store (ChromaDB) | User Query → Embed Query → Search Vector DB → Build Prompt → LLM (GPT-4o) → Final Answer

Module 04: Embeddings

Convert text ⟶ vectors for semantic understanding

Module 05: Vector DBs

Store vectors + enable fast ANN similarity search

Module 06: RAG

Retrieve relevant context ⟶ inject into LLM prompt

KNOWLEDGE CHECK

Quick Quiz — Intermediate P2

1. What is an embedding and why are similar sentences close in vector space?
2. Name 3 vector databases and one reason you'd choose each.
3. Explain the difference between Flat, IVF, and HNSW indexing.
4. Walk through all steps of a RAG pipeline from raw PDF to final answer.

Intermediate Level Complete! 🎓

All 6 Intermediate Modules Covered

✓ 01
Prompt Engineering

✓ 02
LLM APIs

✓ 03
LangChain Basics

✓ 04
Embeddings

✓ 05
Vector Databases

✓ 06
RAG

You Are Ready For Advanced Level! 🚀

- Fine-Tuning LLMs
- Agents & Tool Use
- LLM Evaluation
- LLMOps & Deployment

©2025 Jade Global. All rights reserved.